

IneqMath: Evaluating Mathematical Inequality Proving Ability of LLMs

Based on: *"IneqMath: A Benchmark for Evaluating LLMs on Inequality Proving"*

arXiv: 2506.07927v3

Lecture 10: Large Deep Learning Systems

Outline

1. Motivation: Why Inequality Proving?
2. Task Formalization: Informal Yet Verifiable
3. The INEQMATH Dataset
4. LLM-as-Judge Evaluation Framework
5. Experimental Results on 29 LLMs
6. In-depth Analysis: Scaling Laws & Error Patterns
7. Improvement Strategies
8. Key Takeaways



Why Mathematical Inequality Proving?

- ▶ Inequalities are fundamental to analysis, optimization, probability theory
- ▶ Proving an inequality requires:
 - ▶ Intuition for discovering tight bounds
 - ▶ Strategic selection of classical theorems (AM-GM, Cauchy-Schwarz, ...)
 - ▶ Precise symbolic transformations
- ▶ These are hallmarks of **advanced mathematical reasoning**
- ▶ Automating this process would:
 - ▶ Supply ATPs with missing lemmas
 - ▶ Accelerate formal verification
 - ▶ Serve as a demanding testbed for general-purpose reasoners

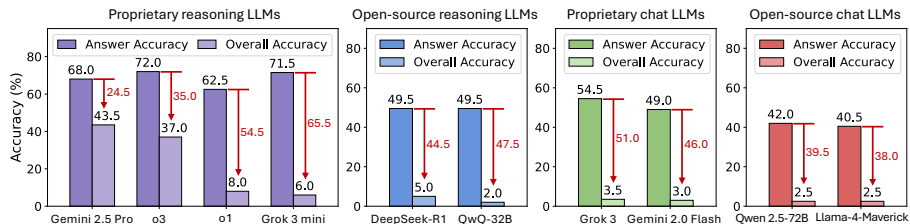


- ▶ Existing datasets fall short:
 - ▶ General ATP collections (MiniF2F, ProofNet): few inequalities
 - ▶ Synthetic datasets (INT, AIPS): scale but lack structural diversity
 - ▶ Curated collections (ChenNEQ): too small for extensive training
- ▶ Most existing datasets adopt **fully formal** representation (Lean, Isabelle)
- ▶ LLMs trained on vast natural language corpora exhibit strong **informal reasoning**
- ▶ Gap: no large-scale benchmark for **informal** inequality proving

Key Contributions

1. **Informal task formulation:** Decompose inequality proving into two verifiable subtasks (bound estimation & relation prediction)
2. **IneqMath benchmark:** Expert-curated Olympiad-level inequalities with step-wise solutions and theorem annotations
3. **LLM-as-judge framework:** Final-answer judge + four step-wise judges for rigorous evaluation
4. **Systematic evaluation:** 29 leading LLMs — exposes a gap between finding answers and constructing rigorous proofs

Key Results at a Glance



Final-answer accuracy vs. overall accuracy across leading LLMs.

Overall accuracy (answer + step soundness) is substantially lower for **all** model types.



Task Formalization: Informal Yet Verifiable

- ▶ Traditional formalization: (S_0, I, P) in formal proof assistants
 - ▶ Demands expertise in specialized tools
 - ▶ Informal proofs are more intuitive but hard to verify automatically
- ▶ Proposed approach: reformulate into two **verifiable** subtasks
 1. **Bound Estimation** — find the optimal constant C
 2. **Relation Prediction** — identify the correct relation $(>, \geq, =, \leq, <)$
- ▶ Both tasks:
 - ▶ Presented in natural language + $\text{\LaTeX}$
 - ▶ Solved step-by-step by an LLM
 - ▶ Final answers automatically checkable

Subtask 1: Bound Estimation

A bound estimation problem instance is a triple:

$$\Pi_{\text{bound}} = (f(\mathbf{x}), g(\mathbf{x}), \mathcal{D}), \quad \text{where } \mathcal{D} \subseteq \mathbb{R}^n$$

- ▶ $f, g : \mathcal{D} \rightarrow \mathbb{R}$ are expressions over $\mathbf{x} = (x_1, \dots, x_n)$
- ▶ $g(\mathbf{x}) > 0, \forall \mathbf{x} \in \mathcal{D}$
- ▶ Goal: determine the extremal constant

$$C^* = \sup\{C \in \mathbb{R} : f(\mathbf{x}) \geq C g(\mathbf{x}), \forall \mathbf{x} \in \mathcal{D}\}$$

or

$$C^* = \inf\{C \in \mathbb{R} : f(\mathbf{x}) \leq C g(\mathbf{x}), \forall \mathbf{x} \in \mathcal{D}\}$$

Example: In $a + b \geq C\sqrt{ab}$ for $\forall a, b > 0$, find the largest C .



Subtask 2: Relation Prediction

A relation prediction problem instance is a triple:

$$\Pi_{\text{rel}} = (f(\mathbf{x}), g(\mathbf{x}), \mathcal{D})$$

- ▶ $f(\mathbf{x})$ and $g(\mathbf{x})$ are expressions over $\mathbf{x}$ in domain $\mathcal{D} \subseteq \mathbb{R}^n$
- ▶ Goal: find the relation between $f(\mathbf{x})$ and $g(\mathbf{x})$
 - ▶ Options: $>$, $\geq$, $=$, $\leq$, $<$, or none of the above

Example: Given $f(\mathbf{x}) = a + b$ and $g(\mathbf{x}) = 2\sqrt{ab}$,
identify the relation for $\forall a, b > 0$.

Answer: $\geq$ (by AM-GM inequality)



Training Example: Bound Problem

Question: Find the maximal constant C such that for all real numbers a, b, c :

$$\sqrt{a^2 + (1-b)^2} + \sqrt{b^2 + (1-c)^2} + \sqrt{c^2 + (1-a)^2} \geq C$$

Solution: Applying Minkowski's Inequality:

$$\text{LHS} \geq \sqrt{(a+b+c)^2 + (3-a-b-c)^2}$$

Denoting $a+b+c = x$:

$$= \sqrt{2\left(x - \frac{3}{2}\right)^2 + \frac{9}{2}} \geq \sqrt{\frac{9}{2}} = \boxed{\frac{3\sqrt{2}}{2}}$$

Theorem used: Minkowski's Inequality



Training Example: Relation Problem

Question: Let $a, b, c > 0$ with $abc = 1$. Determine the relation:

$$\frac{b+c}{\sqrt{a}} + \frac{c+a}{\sqrt{b}} + \frac{a+b}{\sqrt{c}} \quad (\quad) \quad \sqrt{a} + \sqrt{b} + \sqrt{c} + 3$$

Solution (using AM-GM):

$$\text{LHS} \geq 2 \left(\sqrt{\frac{bc}{a}} + \sqrt{\frac{ca}{b}} + \sqrt{\frac{ab}{c}} \right) \geq 2(\sqrt{a} + \sqrt{b} + \sqrt{c})$$

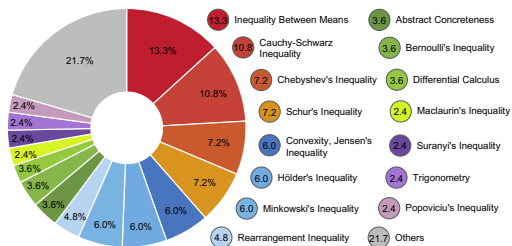
$$\boxed{\geq} \sqrt{a} + \sqrt{b} + \sqrt{c} + 3\sqrt[6]{abc} = \sqrt{a} + \sqrt{b} + \sqrt{c} + 3$$

Answer: $\geq$ **Theorem used:** AM-GM Inequality

- ▶ First large-scale dataset of Olympiad-level inequalities in **informal language**
- ▶ **Test set:** 200 original problems crafted by IMO-level medalists
 - ▶ Reviewed by separate expert group; validated for solvability & soundness
 - ▶ Easier problems excluded (repurposed for development)
- ▶ **Training corpus:** 1,252 problems from advanced textbooks
 - ▶ Automatically rephrased by LLMs, then reviewed by human experts
 - ▶ Up to 4 step-wise solution paths per problem
 - ▶ 76.8% annotated with 83 named theorems across 29 categories
- ▶ **Development set:** 100 problems with public ground truth

IneqMath: Key Statistics

Statistic	Number	Bnd.	Rel.
Theorem categories	29	-	-
Named theorems	83	-	-
Training problems	1252	626	626
- With theorem annotations	962	482	480
- With solution annotations	1252	626	626
- Avg. solutions per problem	1.05	1.06	1.05
- Max solutions per problem	4	4	4
Dev problems	100	50	50
Test problems	200	96	104



Distribution of theorem categories

Datasets	Data Source		Annotation		Problem & Evaluation		
	Training	Test/Dev	#Thm	Soln	Category	Format	Evaluation
INT	Synthesized	Synthesized	35	✓	Proof	Formal	Symbolic DSL
AIPS	Synthesized	—	8	✓	Proof	Formal	Symbolic DSL
MiniF2F	—	Autoformalization	—	—	Proof	Formal	Lean
ProofNet	—	Autoformalization	—	—	Proof	Formal	Lean
leanWorkbook	Autoform.	Autoform.	—	—	Proof	Formal	Lean
Proof or Bluff	—	Data compil.	—	—	Proof	Informal	Human
CHAMP	—	Autoform.	—	—	Open	Informal	Human
Putnam Axiom	—	Data compil.	—	—	Open	Informal	Ans. check
IneqMath (Ours)	Expert	Expert	83	✓	MC, Open	Informal	LLM-as-judge

INEQMATH provides expert-annotated data, 83 named theorems, step-wise solutions, and LLM-as-judge evaluation.



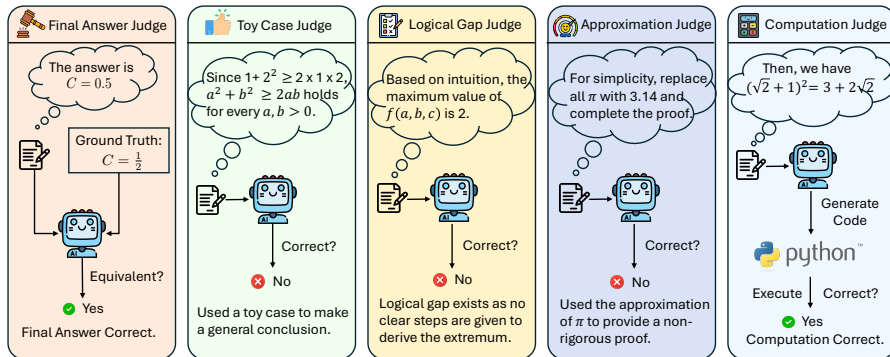
IneqMath: Unique Features

1. Expert-curated training **and** test sets (IMO-level medalists)
2. Rich annotations: step-wise solutions + 83 grounded theorems (29 categories)
3. Informal, accessible format: bound estimation & relation prediction
4. Evaluated via **LLM-as-judge** — bridges formal proof systems and intuitive reasoning
5. Contamination mitigation: novel test problems, memorization probe conducted



- ▶ Producing the correct final answer is **insufficient**
- ▶ The reasoning process itself must be sound
- ▶ Traditional methods fall short:
 - ▶ Expert annotation: accurate but prohibitively labor-intensive
 - ▶ String matching / value equivalence: cannot capture step-wise correctness
- ▶ Need: scalable, fine-grained evaluation of both answers **and** proof steps

LLM-as-Judge Framework: Architecture



A solution is considered correct only if it passes **all five judges**.

Final Answer Judge

- ▶ LLM solutions contain multiple reasoning steps + concluding answer
- ▶ Final answer may vary in phrasing, structure, or numeric format
 - ▶ E.g., $C = \frac{1}{\sqrt{2}}$ and $C = \frac{\sqrt{2}}{2}$ are mathematically equivalent
- ▶ Two-stage approach:
 1. Identify the concluding sentence with the predicted answer
 2. Perform robust **mathematical equivalence checking**
- ▶ Achieves **perfect** agreement with human labels ($F1 = 1.00$)

Step-wise Judge 1: Toy Case Judge

- ▶ **Problem:** LLMs often generalize incorrectly from specific numerical examples
 - ▶ E.g., claiming an inequality holds universally because it holds for $a = 1, b = 2, c = 3$
- ▶ Inequality problems require reasoning over **continuous domains** (e.g., all $a, b, c > 0$)
- ▶ Specific instances are insufficient for a valid proof
- ▶ **Toy Case Judge:** Detects unjustified generalization from toy examples
- ▶ Prompts an LLM to flag conclusions based solely on specific instances without broader justification



Step-wise Judge 2: Logical Gap Judge

- ▶ **Problem:** LLMs often skip key reasoning steps or assert conclusions without support
 - ▶ E.g., stating an optimal bound without derivation
- ▶ Multi-step derivations need explicit justification:
 - ▶ Algebraic manipulation
 - ▶ Constrained optimization
 - ▶ Functional transformations
- ▶ **Logical Gap Judge:** Flags missing transitions, unjustified claims, and vague derivations
- ▶ Most common error type: **85.0%** average failure rate across models



Step-wise Judge 3: Numerical Approximation Judge

- ▶ **Problem:** Inequality problems demand **exact** symbolic reasoning
 - ▶ Replacing $\sqrt{2}$ with 1.414 can compromise mathematical rigor
- ▶ Many LLM solutions resort to numeric approximations in intermediate steps
- ▶ Leads to inaccurate or non-generalizable conclusions
- ▶ **Numerical Approximation Judge:**
 - ▶ Flags inappropriate use of numeric approximations
 - ▶ Specifically when they affect derivations or final answers
 - ▶ Approximations for intuition or side remarks are **permitted**



Step-wise Judge 4: Numerical Computation Judge

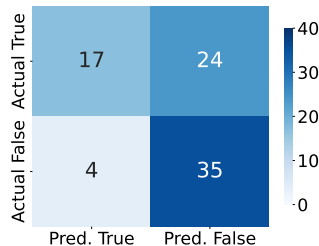
- ▶ **Problem:** Arithmetic accuracy is crucial for overall correctness
 - ▶ E.g., evaluating $\frac{27}{2}$ or summing rational terms
- ▶ **Numerical Computation Judge:**
 - ▶ Verifies arithmetic steps once variables are instantiated
 - ▶ Extracts numerical expressions $\rightarrow$ converts to Python code $\rightarrow$ evaluates
 - ▶ Uses floating-point arithmetic within small tolerance
 - ▶ Encourages use of SymPy for symbolic precision
- ▶ Least frequent error type but still significant: **6.8%** average failure rate

Judge Performance on Development Set

LLM-as-Judge	Accuracy	Precision	Recall	F1
Final Answer Judge	1.00	1.00	1.00	1.00
Toy Case Judge	0.91	0.86	0.97	0.91
Logical Gap Judge	0.96	0.95	0.98	0.96
Num. Approximation Judge	0.96	0.95	0.98	0.96
Num. Computation Judge	0.71	0.68	0.98	0.80
Average	0.91	0.89	0.98	0.93

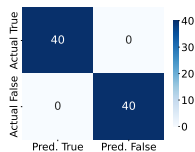
Strong alignment with human annotations (overall F1 = 0.93).

- ▶ Tested a heuristic *LLM-as-judge* baseline
- ▶ Single general-purpose LLM holistically assesses solution correctness
- ▶ Result: **poor agreement** with human labels
- ▶ Decomposing evaluation into targeted sub-problems allows LLMs to serve as **reliable** evaluators

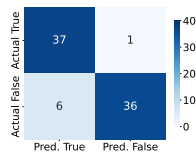


Confusion matrix for holistic judge baseline

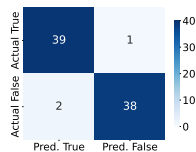
Confusion Matrices: Five Specialized Judges



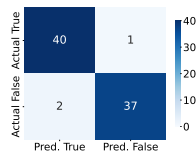
(a) Final answer



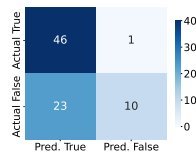
(b) Toy case



(c) Logical gap



(d) Num. approx.



(e) Num. computation

All judges exhibit strong agreement with human labels.



Experimental Setup

- ▶ **29 leading LLMs** evaluated on INEQMATH test set (200 problems)
- ▶ Two categories:
 - ▶ General-purpose chat models (open-source & proprietary)
 - ▶ Specialized reasoning LLMs
- ▶ **Zero-shot** prompting: problem statement + instruction for rigorous steps
- ▶ Three metrics:
 - ▶ *Answer Acc*: final answer correctness (final-answer judge)
 - ▶ *Step Acc*: step-wise correctness (four step-wise judges)
 - ▶ *Overall Acc*: correct answer **AND** flawless steps (all five judges)



Main Results: The Big Picture

- ▶ **Finding 1:** Reasoning LLMs achieve higher final-answer accuracy
 - ▶ o1: 62.5% Answer Acc vs. GPT-4o: 37.5%
 - ▶ Grok 3 mini: 71.5% Answer Acc
- ▶ **Finding 2:** Step-wise scrutiny reveals **dramatic** performance drop
 - ▶ Grok 3 mini: 71.5% → 6.0% Overall Acc (−**65.5%**)
 - ▶ o3-mini: 62.5% → 9.5% Overall Acc (−**53.0%**)
- ▶ **Finding 3:** Even top models achieve very low Overall Acc
 - ▶ o1: 8.0% — o3: 21.0% — o4-mini: 15.5%

Main Results: Selected Models

Models	Overall Acc (↑)			Answer Acc (↑)		
	All	Bnd.	Rel.	All	Bnd.	Rel.
<i>Open-source Chat LLMs</i>						
Qwen2.5-72B	2.5	3.1	1.9	42.0	51.0	33.7
Llama-4-Maverick	2.5	2.1	2.9	40.5	45.8	35.6
<i>Proprietary Chat LLMs</i>						
GPT-4o	3.0	2.1	3.8	37.5	40.6	34.6
Grok 3	3.5	4.2	2.9	54.5	66.7	43.3
<i>Reasoning LLMs</i>						
DeepSeek-R1	5.0	4.2	5.8	49.5	67.7	32.7
o1	8.0	7.3	8.7	62.5	62.5	62.5
o3-mini	9.5	7.3	11.5	62.5	69.8	55.8
o4-mini	15.5	14.6	16.3	65.0	63.5	66.3
o3	21.0	18.8	23.1	37.0	30.2	43.3
Gemini 2.5 Pro	6.0	7.3	4.8	7.0	8.3	5.8

Step-wise Error Breakdown (Average Across Models)

Error Category	Avg. Failure Rate	Severity
Logical Gaps	85.0%	Highest
Toy Case (unjustified generalization)	59.7%	High
Numerical Approximations	26.9%	Moderate
Numerical Computation	6.8%	Lower

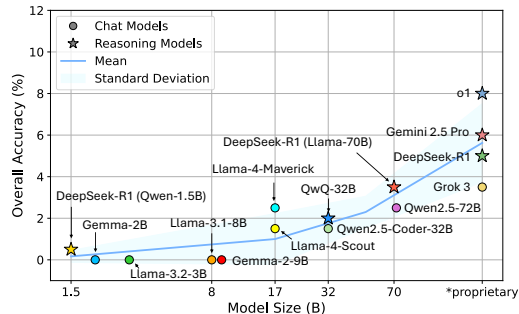
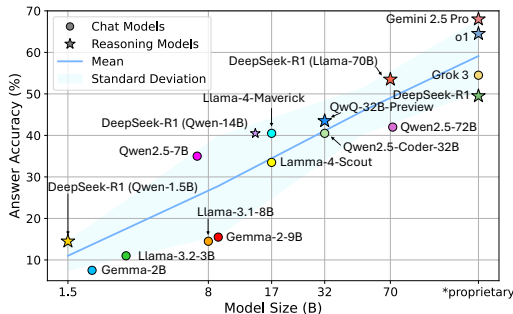
- ▶ **Logical gaps** dominate: LLMs skip key steps or assert without derivation
- ▶ **Toy cases** are the second most common: generalizing from specific examples
- ▶ These errors often undermine proofs **even when the final answer is correct**



Interpretation: Answer Accuracy $\neq$ Proof Correctness

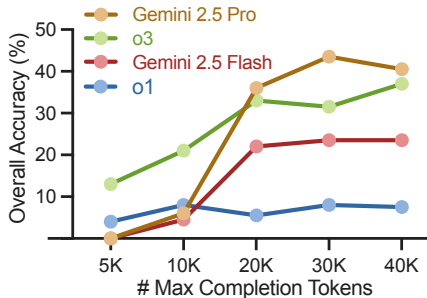
- ▶ High answer accuracy masks fundamental reasoning flaws
- ▶ LLMs can often “guess” or heuristically find the right bound/relation
- ▶ But constructing a **mathematically rigorous derivation** remains a major challenge
- ▶ The gap between *Answer Acc* and *Overall Acc* ranges from 16–65.5%
- ▶ This highlights **fragile deductive chains** in current LLMs

Scaling Law: Model Size



- Larger models $\Rightarrow$ steady improvement in *Answer Acc*
- But the trend **does not hold** for *Overall Acc* — the curve flattens
- Increased model size alone is insufficient to eliminate step-wise reasoning errors

- ▶ Extended reasoning chains are a common strategy for complex problems
- ▶ Varied max completion tokens for reasoning LLMs
- ▶ **Result:** Performance gains saturate beyond $\sim 20K$ tokens
- ▶ o1: 8.0% Overall Acc unchanged from 5K to 40K tokens
- ▶ o3: saturates around 31%
- ▶ Merely increasing computational budget yields **diminishing returns**



Test-time scaling for reasoning LLMs

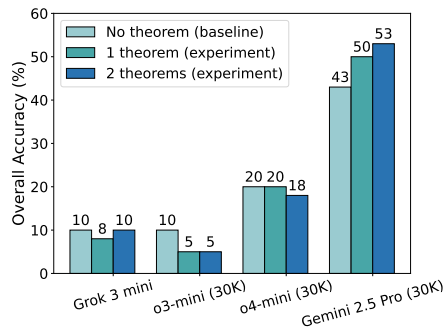


Summary of Scaling Findings

- ▶ **Model size scaling:**
 - ▶ Improves answer accuracy but **not** proof correctness
 - ▶ o1 achieves only 8.0% overall accuracy despite being among the largest
- ▶ **Test-time computation scaling:**
 - ▶ Diminishing returns for achieving rigorous step-wise proofs
 - ▶ Need **more than just longer thought processes**
- ▶ Current scaling approaches are **insufficient** for robust deductive reasoning
- ▶ Need fundamentally different strategies

Improvement Strategy 1: Theorem-guided Reasoning

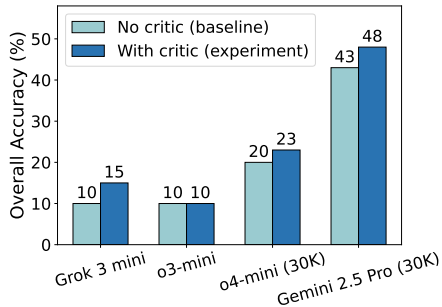
- ▶ Provide top- k most frequent theorems from training corpus as hints
- ▶ Tested on 40-problem test subset
- ▶ **Weaker models:** 1–2 theorems **decrease** performance
 - ▶ Likely misapplication or distraction
- ▶ **Stronger models** (Gemini 2.5 Pro): **benefit** from theorem hints
- ▶ Golden theorems improve o3-mini overall accuracy by up to **+11%**
- ▶ Need sophisticated theorem-retrieval mechanisms (RAG)



Model performance with retrieved theorem hints

Improvement Strategy 2: Self-Critique Refinement

- ▶ Allow LLM to critique and revise its own reasoning
- ▶ One round of self-critique on 40 test problems
- ▶ **Consistent improvement** across models:
 - ▶ Gemini 2.5 Pro: 43% → 48% overall accuracy (+5%)
- ▶ Supervision-free method
- ▶ Enhances logical rigor and solution quality
- ▶ Promising direction for further exploration



Model performance with self-critique feedback



Comparison of Improvement Strategies

- ▶ **Theorem-guided reasoning:**
 - ▶ Up to +11% overall accuracy with golden theorems
 - ▶ Requires careful theorem selection; irrelevant hints hurt weaker models
 - ▶ Future: sophisticated retrieval-augmented generation (RAG)
- ▶ **Self-critique refinement:**
 - ▶ +5% absolute improvement for strong models
 - ▶ Supervision-free; no external knowledge needed
 - ▶ Complementary to theorem-guided approaches
- ▶ Both strategies suggest that **structured reasoning support** is more effective than simply scaling model size or compute



Related Work: Datasets

- ▶ **Formal ATP datasets:** MiniF2F, ProofNet, FormalMATH, leanWorkbook
 - ▶ Encoded in Lean/Isabelle; few inequalities
- ▶ **Synthetic datasets:** INT (35 theorems), AIPS (8 theorems)
 - ▶ Template-based generation $\Rightarrow$ limited structural diversity
- ▶ **Informal reasoning datasets:** Proof or Bluff, CHAMP, Putnam Axiom, LiveMathBench
 - ▶ Lack annotated solutions, theorem references, or training data
- ▶ **IneqMath** fills the gap: 1,252 training problems with theorem annotations and step-wise solutions in informal language



Related Work: Methods

- ▶ **Traditional ATPs:** Operate within formal systems (Lean, Isabelle)
 - ▶ Model theorem proving as MDP; train LLMs for tactic prediction
 - ▶ E.g., Goedel-Prover, InternLM2.5-StepProver
- ▶ **Tree-search methods:** Navigate search space of premises in formal frameworks
- ▶ **LLM-as-judge for math:**
 - ▶ Existing step-level judges lack granularity for nuanced reasoning flaws
 - ▶ EIC-Math: broad error categories but misses subtle issues
- ▶ **This work:** Informal yet verifiable framework with fine-grained evaluation

Conclusion

1. **Informal yet verifiable** task formulation: bound estimation + relation prediction
2. **IneqMath benchmark**: 200 test + 100 dev + 1,252 training problems with step-wise solutions and 83 theorem annotations
3. **LLM-as-judge** framework ($F1 = 0.93$): final-answer judge + four step-wise judges
4. **Critical finding**: LLMs achieve up to 71.5% answer accuracy but plummet to $\leq 21\%$ overall accuracy under step-wise scrutiny
5. Scaling model size or test-time compute yields **limited gains** in proof correctness
6. Promising directions: theorem-guided reasoning (+11%) and self-critique (+5%)

Key Takeaways

- ▶ **Answer accuracy $\neq$ proof correctness:** A correct final answer does not imply rigorous reasoning
- ▶ **Fragile deductive chains:** Even SOTA reasoning LLMs produce logically flawed proofs
- ▶ **Dominant error:** Logical gaps (85%) and toy-case overgeneralization (60%)
- ▶ **Scaling limits:** Neither model size nor test-time compute sufficiently address step-wise errors
- ▶ **Structured reasoning support** (theorem hints, self-critique) is more promising than brute-force scaling
- ▶ **Open challenge:** Bridging the gap between heuristic answer-finding and rigorous mathematical proof construction